디지털 설계 퀴즈 예상문제 (11/10) — 정답 포함

2026년 11월 8일 작성 · 노트의 퀴즈 범위 기준 · 36문제 · 정답 포함 버전

안내

노트에 적힌 퀴즈 범위 7개 묶음에서 예상문제 36개를 뽑았어요. 밑줄 친 SIPO/PISO와 설계 가이드·Latch는 문제를 더 넣었어요.

강의에서 퀴즈는 개념 서술형("왜 이렇게 설계하나?")과 설계 문제가 나오고, 주관식도 있을 수 있다고 했어요. 그래서 문제 유형을 세 가지로 나눴어요.

표시	유형	준비 포인트
[개념]	서술형	이유까지 2~3문장으로 설명할 수 있어야 해요
[해석]	코드 읽기	합성 결과나 시뮬레이션 값을 예측해요
[설계]	코드 작성	손으로 Verilog를 써 봐요

이 버전은 문제마다 바로 아래에 정답과 해설을 붙였어요. 복습할 때 보는 용도예요.

1. 조합 논리: MUX, Decoder

1-1. [설계] 8비트 데이터 4개(`a`, `b`, `c`, `d`) 중 하나를 2비트 `sel`로 고르는 4:1 MUX를 작성하세요. (1) `assign`과 조건 연산자로 한 번, (2) `always` 블록과 `case`로 한 번 작성합니다.

정답·해설

```
// (1) assign + 조건 연산자
assign y = (sel == 2'b00) ? a :
           (sel == 2'b01) ? b :
           (sel == 2'b10) ? c : d;

// (2) always + case
always @(*) begin
  case (sel)
    2'b00:   y = a;
    2'b01:   y = b;
    2'b10:   y = c;
    default: y = d;
  endcase
end
```

(2)에서 `y`는 `reg`(SystemVerilog면 `logic`)로 선언해요.

1-2. [해석] 아래 코드를 합성하면 MUX 외에 의도하지 않은 소자가 생깁니다. 무엇이 생기고 왜 생기는지, 어떻게 고치는지 쓰세요.

```
always @(*) begin
  case (sel)
    2'b00: y = a;
    2'b01: y = b;
    2'b10: y = c;
  endcase
end
```

> **정답·해설**
>
> **Latch**가 생겨요. `sel = 2'b11` 일 때 `y` 에 값을 주지 않아서 이전 값을 기억해야 하기 때문이에요. `default: y = 1'b0;` (또는 `d`)를 넣거나, `case` 앞에 `y = 0;` 처럼 기본값을 먼저 할당해서 고쳐요.

1-3. [설계] enable 입력이 있는 2:4 디코더의 진리표를 그리고 Verilog로 작성하세요. `en` 이 0이면 출력은 모두 0입니다.

> **정답·해설**
>
> | en | a[1:0] | y[3:0] |
> |---|---|---|
> | 0 | xx | 0000 |
> | 1 | 00 | 0001 |
> | 1 | 01 | 0010 |
> | 1 | 10 | 0100 |
> | 1 | 11 | 1000 |
>
> ```
> always @(*) begin
>   if (!en) y = 4'b0000;
>   else case (a)
>     2'b00: y = 4'b0001;
>     2'b01: y = 4'b0010;
>     2'b10: y = 4'b0100;
>     2'b11: y = 4'b1000;
>   endcase
> end
> // 한 줄 버전: assign y = en ? (4'b0001 << a) : 4'b0000;
> ```

1-4. [해석] 아래 코드는 어떤 회로입니까? `en = 1`, `a = 3'b101` 일 때 `y` 값은?

```
assign y = en ? (8'b0000_0001 << a) : 8'b0;
```

> **정답·해설**
>
> enable이 있는 **3:8 디코더**예요. `a = 5` 이므로 1을 5칸 왼쪽으로 밀어 `y = 8'b0010_0000` 이에요.

1-5. [개념] 조합 논리와 순차 논리의 차이를 설명하세요. 조합 논리를 `always` 로 쓸 때 sensitivity list에 `@(*)` 를 쓰는 이유도 쓰세요.

> **정답·해설**

조합 논리는 출력이 **현재 입력만으로** 결정되고 기억 소자가 없어요. 순차 논리는 현재 입력과 **저장된 상태(플립플롭)**로 출력이 결정되고 클럭에 맞춰 바뀌어요. `@(*)`는 블록 안에서 읽는 모든 신호를 자동으로 넣어 줘요. 신호를 빠뜨려서 생기는 시뮬레이션과 합성 결과의 불일치를 막아요.

2. 순차 논리: Counter, tick_gen

2-1. [설계] 100 MHz 클럭으로 1 kHz `tick`(클럭 1주기 폭의 펄스)을 만드는 `tick_gen`을 작성하세요. 카운터의 최대값과 필요한 비트 수도 구하세요.

정답·해설

100 MHz ÷ 1 kHz = 100,000이므로 100,000클럭마다 한 번 `tick`을 내요. 카운터는 0~99,999를 세고 **최대값은 99,999**예요. 2^16 = 65,536 < 100,000 ≤ 2^17 = 131,072이므로 **17비트**가 필요해요.

```
module tick_gen #(parameter DIV = 100_000) (
  input clk, input rst,
  output reg tick
);
  reg [16:0] cnt;
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      cnt <= 0;  tick <= 1'b0;
    end else if (cnt == DIV - 1) begin
      cnt <= 0;  tick <= 1'b1;
    end else begin
      cnt <= cnt + 1;  tick <= 1'b0;
    end
  end
endmodule
```

2-2. [설계] 0부터 9까지 세는 BCD 업 카운터를 작성하세요. 동기 리셋 `rst`와 카운트 허용 `en`이 있고, 9에서 0으로 넘어갈 때 `carry`를 1로 냅니다.

정답·해설

```
always @(posedge clk) begin
  if (rst)      cnt <= 4'd0;
  else if (en) begin
    if (cnt == 4'd9) cnt <= 4'd0;
    else             cnt <= cnt + 4'd1;
  end
end
assign carry = en && (cnt == 4'd9);
```

2-3. [해석] 아래 카운터는 몇 진 카운터입니까? `tick`은 몇 클럭마다 한 번 나옵니까?

```
always @(posedge clk) begin
  if (cnt == 10) cnt <= 0;
  else           cnt <= cnt + 1;
end
assign tick = (cnt == 10);
```

> **정답·해설**
> 0부터 10까지 세므로 **11진 카운터**예요. `tick`은 **11클럭마다 한 번**, 1클럭 폭으로 나와요. 리셋이 없어서 시뮬레이션에서는 초기값이 X로 남는 점도 지적할 수 있어요.

2-4. [개념] 느린 동작이 필요할 때, 분주한 클럭(`clk_div`)을 다른 플립플롭의 클럭으로 쓰는 대신 `tick`을 enable로 쓰는 이유를 설명하세요.

> **정답·해설**
> 분주 클럭은 플립플롭 출력이라 원래 클럭보다 늦게 도착해요(skew). 그러면 새로운 클럭 도메인이 생겨 CDC와 타이밍 분석이 복잡해지고, FPGA에서는 전용 클럭 배선도 쓰지 못해요. `tick`을 enable로 쓰면 모든 플립플롭이 **같은 `clk` 하나**로 동작해서 완전한 동기식 설계가 유지돼요.

3. F/F, Synchronizer, SIPO/PISO

3-1. [설계] D 플립플롭을 (1) 비동기 리셋, (2) 동기 리셋으로 각각 작성하세요. 두 코드의 차이가 나타나는 곳을 표시하세요.

> **정답·해설**
> 차이는 **sensitivity list에 `rst`가 있느냐**예요.
>
> ```
> // (1) 비동기 리셋: 클럭과 상관없이 즉시 리셋
> always @(posedge clk or posedge rst)
>   if (rst) q <= 1'b0;
>   else     q <= d;
>
> // (2) 동기 리셋: 클럭 엣지에서만 리셋
> always @(posedge clk)
>   if (rst) q <= 1'b0;
>   else     q <= d;
> ```

3-2. [설계·개념] 비동기 입력 `async_in`을 `clk`에 맞추는 2-FF Synchronizer를 작성하세요. 플립플롭을 두 단 쓰는 이유와, 첫 번째 플립플롭 출력을 바로 쓰면 안 되는 이유를 설명하세요.

> **정답·해설**

```
reg sync1, sync2;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    sync1 <= 1'b0;  sync2 <= 1'b0;
  end else begin
    sync1 <= async_in;
    sync2 <= sync1;
  end
end
assign sync_out = sync2;
```

비동기 입력은 setup/hold를 위반할 수 있어서 첫 번째 플립플롭이 **메타스테이블** 상태가 될 수 있어요. 두 번째 플립플롭은 한 클럭 동안 안정될 시간을 준 뒤 값을 받아서, 안정된 값을 얻을 확률이 크게 올라가요(MTBF 증가). 첫 번째 출력을 바로 쓰면 아직 불안정한 값이 여러 로직으로 퍼져 서로 다른 값으로 해석될 수 있어요.

3-3. [설계] 8비트 SIPO를 작성하세요. 직렬 입력 `din`을 MSB부터 받고, 8비트가 모이면 병렬 출력 `dout[7:0]`을 내면서 `valid`를 1클럭 동안 1로 냅니다.

정답·해설

```
reg [7:0] sr;
reg [2:0] cnt;
always @(posedge clk or posedge rst) begin
  if (rst) begin
    sr <= 0;  cnt <= 0;  dout <= 0;  valid <= 1'b0;
  end else begin
    sr    <= {sr[6:0], din};      // MSB부터 들어옴
    cnt   <= cnt + 1;             // 7 다음 0으로 자동 순환
    valid <= (cnt == 3'd7);
    if (cnt == 3'd7) dout <= {sr[6:0], din};
  end
end
```

3-4. [설계] 8비트 PISO를 작성하세요. `load`가 1이면 `din[7:0]`을 병렬로 저장하고, 그 뒤 매 클럭 MSB부터 한 비트씩 `dout`으로 내보냅니다.

정답·해설

```
reg [7:0] sr;
always @(posedge clk or posedge rst) begin
  if (rst)       sr <= 8'd0;
  else if (load) sr <= din;
  else           sr <= {sr[6:0], 1'b0};  // 왼쪽으로 시프트
end
assign dout = sr[7];                     // MSB부터 출력
```

3-5. [해석] 아래 시프트 레지스터에 `din`으로 1, 0, 1, 1을 차례로 넣었습니다. 네 번째 클럭 직후 `sr` 값은? (초기값 `4'b0000`)

```
always @(posedge clk)
  sr <= {sr[2:0], din};
```

> **정답·해설**
> 1 → `0001`, 0 → `0010`, 1 → `0101`, 1 → `1011`. 답은 **`4'b1011`**이고, 처음 들어간 비트가 MSB에 있어요.

3-6. [개념] SIPO와 PISO는 실제로 어디에 쓰입니까? 두 가지 예와 쓰는 이유를 쓰세요.

> **정답·해설**
> UART·SPI 같은 **직렬 통신**에 써요. 송신은 PISO로 병렬 데이터를 한 비트씩 내보내고, 수신은 SIPO로 모아 병렬로 만들어요. 칩 사이 **핀 수와 배선을 줄이면서** 칩 내부는 병렬로 빠르게 처리하기 위해서예요.

4. FSM, Pattern Detector

4-1. [개념] Moore FSM과 Mealy FSM의 차이를 출력이 결정되는 방식, 출력 타이밍, 필요한 상태 수 측면에서 설명하세요.

> **정답·해설**

구분	Moore	Mealy
출력 결정	현재 상태만	현재 상태 + 현재 입력
출력 타이밍	클럭 뒤 상태가 바뀐 다음 (1클럭 늦음), 글리치 없이 안정	입력이 바뀌면 바로 반응, 입력 글리치가 출력에 전달될 수 있음
상태 수	보통 더 많음 (`1011` 검출: 5개)	보통 더 적음 (`1011` 검출: 4개)

4-2. [설계] 입력 `din`에서 패턴 `1011`을 검출하는 Moore FSM을 설계하세요. 패턴이 겹쳐도(overlap) 검출합니다. 상태도를 그리고, 상태 레지스터·다음 상태 로직·출력 로직을 나눈 구조로 작성하세요.

> **정답·해설**
> 상태는 지금까지 맞은 부분 패턴으로 정해요. S0(없음), S1(`1`), S2(`10`), S3(`101`), S4(`1011`, 출력 1)이에요.

현재 상태	din = 0	din = 1	출력
S0	S0	S1	0
S1 (`1`)	S2	S1	0
S2 (`10`)	S0	S3	0
S3 (`101`)	S2	S4	0
S4 (`1011`)	S2	S1	1

> S3에서 0이 오면 `1010`의 끝 `10`이 남아 S2로 가요. S4에서 1이 오면 끝 `1`이 남아 S1, 0이 오면 `10`이 남아 S2로 가요. 이것이 overlap 처리예요.

```
localparam S0 = 3'd0, S1 = 3'd1, S2 = 3'd2, S3 = 3'd3, S4 = 3'd4;
reg [2:0] state, next;

// 상태 레지스터 (순차)
always @(posedge clk or posedge rst)
  if (rst) state <= S0;
  else     state <= next;

// 다음 상태 로직 (조합)
always @(*) begin
  case (state)
    S0:      next = din ? S1 : S0;
    S1:      next = din ? S1 : S2;
    S2:      next = din ? S3 : S0;
    S3:      next = din ? S4 : S2;
    S4:      next = din ? S1 : S2;
    default: next = S0;
  endcase
end

// 출력 로직 (Moore: 상태만 봄)
assign detect = (state == S4);
```

4-3. [해석] 4-2의 검출기에 `din`으로 `1 0 1 1 0 1 1`을 한 클럭에 한 비트씩 넣었습니다. 검출은 몇 번 일어납니까? Mealy로 만들면 출력 타이밍은 Moore와 어떻게 달라집니까?

정답·해설

상태는 S1 → S2 → S3 → **S4** → S2 → S3 → **S4** 순으로 바뀌어서 **2번** 검출돼요(4번째, 7번째 비트). Moore는 그 비트를 받은 클럭 엣지 **다음 사이클**에 S4가 되면서 출력이 1이 돼요. Mealy는 S3에서 `din = 1`이 들어와 있는 **그 사이클에 바로** 출력이 1이 돼서 Moore보다 1클럭 빨라요.

4-4. [개념] 다음 세 가지를 각각 답하세요.

1. FSM을 상태 레지스터(순차)와 다음 상태 로직(조합)으로 나눠 쓰는 이유는?
2. 상태 인코딩 방식(Binary, One-hot, Gray)의 장단점은?
3. 다음 상태 `case` 문에 `default`를 넣는 이유는?

정답·해설

1. 순차와 조합을 섞으면 의도치 않은 레지스터나 Latch가 생기기 쉬워요. 나누면 읽기·디버깅이 쉽고, 합성 툴이 FSM을 잘 인식해요.
2. Binary는 플립플롭이 가장 적지만(log2 N개) 디코딩 로직이 많아요. One-hot은 플립플롭이 상태 수만큼 필요하지만 로직이 단순하고 빨라서 FPGA에서 많이 써요. Gray는 인접 상태로 갈 때 1비트만 바뀌어 글리치와 전력이 줄어요. 순서대로 진행하는 FSM에 유리해요.
3. 쓰지 않는 상태 코드(예: 3비트 중 5~7)로 잘못 들어가도 초기 상태로 **복구**하고, 조합 블록에서 Latch가 생기는 것을 막기 위해서예요.

5. 설계 가이드: Latch

5-1. **[개념]** 조합 논리 `always` 블록에서 의도치 않은 Latch가 만들어지는 조건 두 가지를 쓰세요.

> **정답·해설**
> (1) `if` 에 `else` 가 없을 때, (2) `case` 에서 모든 경우를 다루지 않고 `default` 도 없을 때예요. 넓게 보면 "어떤 경로에서 출력에 값을 주지 않을 때"라고 쓰면 돼요.

5-2. **[개념]** Latch를 막는 코딩 방법 세 가지를 쓰세요.

> **정답·해설**
> (1) `always` 블록 맨 앞에서 모든 출력에 기본값을 할당해요. (2) `if` 에는 `else` , `case` 에는 `default` 를 반드시 써요. (3) `always_comb` 를 쓰고 합성 리포트의 Latch 경고를 확인해요.

5-3. **[개념]** 동기식 설계에서 Latch를 피해야 하는 이유를 설명하세요.

> **정답·해설**
> Latch는 enable 동안 입력이 그대로 통과하는 **레벨 감지** 소자예요. 그래서 타이밍 분석(STA)이 어렵고, 글리치에 민감하며, 의도치 않은 타이밍 루프가 생길 수 있어요. 동기식 설계의 "클럭 엣지에서만 값이 바뀐다"는 전제가 깨져요.

5-4. **[해석]** 아래 코드에서 Latch가 생기는 신호를 찾고 고치세요.

```
always @(*) begin
  if (en) begin
    y = a;
    z = b;
  end else begin
    y = 1'b0;
  end
end
```

> **정답·해설**
> `z` 에서 Latch가 생겨요. `en = 0` 일 때 `z` 에 값을 주지 않기 때문이에요. `else` 에 `z = 1'b0;` 을 넣거나 블록 맨 앞에 `y = 1'b0; z = 1'b0;` 처럼 기본값을 써서 고쳐요.

5-5. **[개념]** Latch와 플립플롭의 차이를 동작 방식(레벨/엣지) 기준으로 설명하세요.

> **정답·해설**
> Latch는 **레벨 감지**예요. enable(또는 클럭)이 1인 동안 입력이 출력에 그대로 전달되고(투명), 0이 되면 값을 유지해요. 플립플롭은 **엣지 감지**예요. 클럭 상승(또는 하강) 엣지 순간에만 입력을 저장해요.

5-6. [개념] O/X 설계 가이드에 맞으면 O, 틀리면 X를 하세요.

1. 순차 논리에는 non-blocking(`<=`)을 쓴다.
2. 조합 논리 `always` 블록에는 blocking(`=`)을 쓴다.
3. 한 `always` 블록 안에서 `=`와 `<=`를 섞어 써도 괜찮다.
4. 같은 변수를 여러 `always` 블록에서 할당해도 된다.
5. sensitivity list에서 신호를 빠뜨려도 시뮬레이션과 합성 결과는 같다.
6. SystemVerilog의 `always_comb`를 쓰면 Latch가 추론될 때 툴이 경고를 내준다.

> **정답·해설**
>
> 1 **O** · 2 **O** · 3 **X** (예측하기 어렵고 시뮬레이션·합성 불일치 위험) · 4 **X** (다중 드라이버가 되어 합성 오류나 race 발생) · 5 **X** (합성은 sensitivity list를 무시해서 결과가 달라짐) · 6 **O**

6. Blocking/Non-blocking, Race condition

6-1. [개념] Blocking(`=`)과 Non-blocking(`<=`) 할당의 차이를 값이 계산되는 시점과 실제로 바뀌는 시점 기준으로 설명하세요.

> **정답·해설**
>
> Blocking(`=`)은 그 문장을 실행하는 순간 값을 계산하고 **바로 바꿔요**. 다음 문장은 바뀐 값을 봐요. Non-blocking(`<=`)은 오른쪽 값을 그 시점에 계산만 해 두고, 같은 시간대의 다른 문장이 다 실행된 **뒤에 한꺼번에** 바꿔요. 그래서 플립플롭처럼 모든 레지스터가 동시에 갱신되는 동작을 표현해요.

6-2. [해석] 아래 코드는 3단 시프트 레지스터를 의도했습니다. 실제로는 어떤 회로가 만들어지나요? 의도대로 고치세요.

```
always @(posedge clk) begin
  q1 = d;
  q2 = q1;
  q3 = q2;
end
```

> **정답·해설**
>
> `q1 = d`가 바로 반영되고 `q2`, `q3`도 같은 클럭에서 `d`를 받아요. 결국 셋이 같은 값이 되어 **플립플롭 하나**(한 단 지연)처럼 합성돼요. 3단 시프트 레지스터가 되지 않아요.
>
> ```
> always @(posedge clk) begin
>   q1 <= d;
>   q2 <= q1;
>   q3 <= q2;
> end
> ```

6-3. [해석] `a = 1`, `b = 0`에서 클럭 엣지가 한 번 올라왔습니다. (1)과 (2) 각각 엣지 뒤 `a`, `b` 값은?

```
// (1)
always @(posedge clk) begin a = b;  b = a;  end
// (2)
always @(posedge clk) begin a <= b; b <= a; end
```

> **정답·해설**

(1) `a = b` 로 `a = 0` 이 되고, `b = a` 가 바뀐 `a` (0)를 받아 **`a = 0, b = 0`** 이에요. (2) 오른쪽 값을 엣지 순간에 먼저 읽어 두므로 두 값이 교환돼서 **`a = 0, b = 1`** 이에요.

6-4. [개념] Race condition이 무엇인지 설명하고, 아래 코드에서 왜 발생하는지 쓰세요. 해결 방법도 쓰세요.

```
always @(posedge clk) a = b;
always @(posedge clk) b = a;
```

정답·해설

Race condition은 **같은 시각에 일어난 이벤트의 처리 순서**에 따라 결과가 달라지는 상황이에요. 두 `always` 블록이 같은 클럭 엣지에서 실행되는데, 시뮬레이터가 어느 블록을 먼저 실행할지는 정해져 있지 않아요. 첫 블록이 먼저면 둘 다 `b` 의 옛 값이 되고, 반대면 둘 다 `a` 의 옛 값이 돼요. `<=` 로 바꾸면 순서와 상관없이 항상 두 값이 교환돼요.

6-5. [개념] 테스트벤치가 DUT 입력을 `@(posedge clk)` 직후 blocking 할당으로 바꾸면 어떤 문제가 생길 수 있나요? 피하는 방법 두 가지를 쓰세요.

정답·해설

DUT의 `always @(posedge clk)` 와 같은 시각에 입력이 바뀌어서, DUT가 바뀌기 전 값을 샘플할지 바뀐 뒤 값을 샘플할지가 시뮬레이터 순서에 따라 달라져요(race). 피하는 방법은 다음과 같아요.

1. `@(negedge clk)` 에서 입력을 바꿔요.
2. Non-blocking(`<=`)으로 구동하거나 `#1` 처럼 약간의 지연 뒤에 바꿔요.
3. SystemVerilog의 clocking block을 써요.

7. Reset, CDC

7-1. [개념] 동기 리셋과 비동기 리셋의 장단점을 비교하세요.

정답·해설

구분	동기 리셋	비동기 리셋
장점	글리치에 강함, 완전 동기라 타이밍 분석이 쉬움	클럭 없이도 즉시 리셋, 데이터 경로에 로직이 추가되지 않음
단점	클럭이 있어야 리셋됨, 리셋 펄스가 클럭 주기보다 길어야 함, 데이터 경로에 로직 추가	리셋 해제가 클럭과 무관해서 recovery/removal 위반과 메타스테이블 위험, 리셋 글리치에 민감

7-2. [설계·개념] "비동기 인가, 동기 해제(Asynchronous assert, Synchronous de-assert)" 리셋 동기화기를 작성하세요. 리셋 해제를 클럭에 맞춰야 하는 이유도 쓰세요.

정답·해설

```
module rst_sync (
  input  clk,
  input  rst_n_in,   // 외부 비동기 리셋 (active low)
  output rst_n_out   // 동기화된 리셋
);
  reg r1, r2;
  always @(posedge clk or negedge rst_n_in) begin
    if (!rst_n_in) begin
      r1 <= 1'b0;  r2 <= 1'b0;   // 인가는 즉시 (비동기)
    end else begin
      r1 <= 1'b1;  r2 <= r1;     // 해제는 2클럭 뒤 (동기)
    end
  end
  assign rst_n_out = r2;
endmodule
```

리셋 해제가 클럭 엣지 근처에서 일어나면 recovery/removal 타이밍을 위반해요. 그러면 어떤 플립플롭은 이번 클럭에, 어떤 것은 다음 클럭에 리셋이 풀려 상태가 어긋나고, 메타스테이블도 생길 수 있어요.

7-3. [개념] CDC(Clock Domain Crossing)는 무엇이고, 처리하지 않으면 어떤 문제가 생기나요?

정답·해설

서로 다른(비동기) 클럭으로 동작하는 회로 사이로 신호가 넘어가는 것이에요. 처리하지 않으면 받는 쪽에서 setup/hold 위반으로 **메타스테이빌리티**가 생겨요. 여러 비트는 비트마다 다른 클럭에 잡혀 **잘못된 값**이 되고, 짧은 펄스는 **놓칠** 수 있어요.

7-4. [개념] 8비트 데이터 버스를 다른 클럭 도메인으로 넘길 때, 비트마다 2-FF Synchronizer를 붙이면 안 되는 이유는? 올바른 방법 두 가지를 쓰세요.

정답·해설

비트마다 동기화기를 통과하는 데 걸리는 클럭 수가 다를 수 있어요. 그래서 한순간 일부 비트는 새 값, 일부는 옛 값이 되어 **존재하지 않는 값**을 읽게 돼요. 올바른 방법은 다음과 같아요.

1. **Handshake(req/ack)**: 데이터는 안정하게 유지하고 제어 신호 하나만 동기화해요.
2. **비동기 FIFO**를 써요.
3. 카운터처럼 1씩 변하는 값은 **Gray code**로 바꿔 넘겨요.

7-5. [개념] 100 MHz 도메인의 1클럭 폭 펄스를 10 MHz 도메인으로 넘기려고 합니다. 2-FF Synchronizer만 쓰면 어떤 문제가 생기고, 어떻게 해결하나요?

정답·해설

10 MHz 클럭 주기는 100 ns이고 펄스 폭은 10 ns예요. 받는 쪽 클럭 엣지 사이에서 펄스가 시작하고 끝나 버려 **아예 샘플되지 않을 수** 있어요. 해결 방법은 다음과 같아요.

1. 펄스를 받는 쪽 클럭 주기보다 길게 늘려요(pulse stretch).
2. **Toggle synchronizer**: 보내는 쪽에서 펄스마다 신호를 토글하고, 동기화한 뒤 받는 쪽에서 엣지를 검출해 펄스로 되돌려요.
3. Handshake를 써요.

7-6. [개념] 비동기 FIFO의 읽기·쓰기 포인터를 다른 도메인으로 넘길 때 Gray code로 바꾸는 이유는?

정답·해설

> Binary 카운터는 한 번 증가할 때 여러 비트가 동시에 바뀔 수 있어요(예: `0111` → `1000` 은 4비트). 그래서 동기화 도중 엉뚱한 값이 잡힐 수 있어요. Gray code는 인접한 값 사이에 **1비트만** 바뀌어요. 잘못 잡혀도 옛 값이나 새 값 중 하나이므로 full/empty 판단이 안전해요.